X-MoE: Enabling Scalable Training for Emerging Mixture-of-Experts Architectures on HPC Platforms

Anonymous Author(s)

Abstract

Emerging expert-specialized Mixture-of-Experts (MoE) architectures, such as DeepSeek-MoE, deliver strong model quality through fine-grained expert segmentation and large top-k routing. However, their scalability is limited by substantial activation memory overhead and costly all-to-all communication. Furthermore, current MoE training systems – primarily optimized for NVIDIA GPUs – perform suboptimally on non-NVIDIA platforms, leaving significant computational potential untapped. In this work, we present X-MoE, a novel MoE training system designed to deliver scalable training performance for next-generation MoE architectures. X-MoE achieves this via several novel techniques, including efficient padding-free MoE training with cross-platform kernels, redundancy-bypassing dispatch, and hybrid parallelism with sequence-sharded MoE blocks. Our evaluation on the Frontier supercomputer, powered by AMD MI250X GPUs, shows that X-MoE scales DeepSeek-style MoEs up to 545 billion parameters across 1024 GPUs – 10x larger than the largest trainable model with existing methods under the same hardware budget, while maintaining high training throughput.

1 Introduction

Large Language Models (LLMs) have become the backbone of modern AI applications, achieving remarkable results across domains such as dialogue systems, code generation, and scientific reasoning [6, 27, 28]. However, training these models at scale remains prohibitively expensive. For example, training models at the scale of GPT-3 or GPT-4 consumes hundreds of thousands of GPU days and incurs billions of dollars in compute cost [1, 3, 34, 36]. As such, reducing the training cost while still achieving high model quality has become a key research challenge.

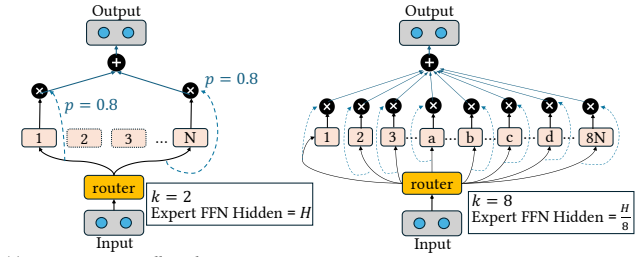
Numerous efforts have been made to improve the training efficiency of LLMs. Among those, Mixture-of-Experts (MoE) have emerged as a promising path to enable sublinear compute with respect to the model parameters, allowing for improved model quality without increased training costs [2, 13, 17, 22, 32]. In contrast to dense models, MoEs sparsely activate model parameters, which allows one to scale to larger model parameters while keeping per-token compute budgets relatively low. Prior works have shown that MoEs can successfully scale to trillions of parameters [13].

More recently, models such as DeepSeek-MoE [10] represent a new class of emerging MoE architectures that depart from earlier designs like GShard [23] and Mixtral-MoE [17]. These models rely on architectural modifications such as fine-grained experts and large top-k routing to allow experts to focus on more distinct contextual concepts, known as expert specialization. As a result, DeepSeek-MoE style models have great potential to accelerate LLM training with low costs, renewing interest in developing scalable and efficient training systems for emerging MoE architectures.

Unfortunately, training expert-specialized MoEs at scale is very challenging. First, existing MoE training systems heavily rely on CUDA-specific implementations for standard MoEs, which are inefficient for expert-specialized MoEs and difficult to port to non-NVIDIA platforms such as AMD Instinct GPUs or Slingshot-based interconnects using ROCm and RCCL. This lack of cross-platform support leads to inflated memory usage and suboptimal performance on heterogeneous HPC systems like Frontier [5] and Aurora [4] (§ 3.1). Second, expert-specialized MoEs introduce a structural shift: they increase the number of routed experts per token and shrink each expert's hidden dimension. This change shifts the memory bottleneck from model parameters to activations, particularly in the dispatch and combine stages. However, existing MoE training systems, such as DeepSpeed-MoE [30], DeepSpeed-TED [33], and Tutel [16], do not effectively address this shifted bottleneck, causing a memory explosion (§ 3.2). Third, many-expert routing significantly increases duplication in communication, especially when multiple experts are selected per token. On platforms with hierarchical interconnects, such as Dragonfly network [20] in Frontier, this results in inefficient use of inter-node bandwidth and causes communication to become a major training efficiency bottleneck as the expert granularity increases (§ 3.3).

To address these challenges, we present system-level optimizations for training emerging MoE architectures on cross-platform, non-NVIDIA hardware. Our analysis reveals that off-the-shelf MoE training systems, designed under the assumption of conventional MoEs and NVIDIA platforms, perform sub-optimally on new MoE architectures and non-NVIDIA hardware. For example, we observe that state-of-the-art MoE frameworks like Tutle [16] and DeepSpeed-MoE [30] achieve < 10 TFLOPS on AMD MI250X GPUs, which is under 10% of their peak performance, whereas Megablocks [14] is highly integrated with NVIDIA Megatron-LM, which does not easily run on AMD hardware. Motivated by these observations, we propose X-MoE, an MoE training system that enables scaling of expert-specialized MoEs across massive GPUs. X-MoE accomplishes this via a combination of techniques, such as padding-free MoE training with cross-platform kernels for improved memory and communication efficiency(§ 4.1), redundancy-bypassing dispatching for communication reduction (§ 4.2), and hybrid parallelism with sequence-sharded MoE blocks (§ 4.3). Unlike prior systems, we achieve this without relying on vendor-specific software stacks like CUDA, and instead rely on portable backends such as Triton [35]. This makes X-MoE backend-agnostic and suitable for future hardware platforms. To our knowledge, X-MoE is the first work that systematically optimizes for both emerging expert-specialized MoEs and the heterogeneity of non-NVIDIA platforms.

We demonstrate our approach on the Frontier supercomputer [5], which consists of AMD MI250X GPUS with Dragonfly architecture [20]. We validate our design through comprehensive experiments. X-MoE enables the training of DeepSeek-style MoEs up-to



(a) Few experts + small top-k routing (b) Fine-grained experts + large top-k routing

Figure 1: Standard vs. expert-specialized MoE architecture.

545B-parameters on 1024 AMD GPUs, which is 10× larger than the largest trainable model under the same hardware budget using existing solutions. Beyond model scale, X-MoE delivers up to 1.42x higher training throughput than state-of-the-art MoE systems, while also outperforming them in both weak and strong scaling. To enhance usability, X-MoE has been integrated with DeepSpeed [31], a popular open-source DL training library, making it accessible for future MoE training workloads.

2 Background

In this part, we introduce important concepts in the MoE literature.

Sparsely Activated MoEs. An MoE layer most commonly replaces the dense FFN in a transformer with a single layer consisting of a set of experts [13, 32]. During training, a token is dynamically routed to k experts based on scores computed by a gating function. This is done in four steps. First, the gating function is applied to the input tokens, generating a mapping of which token should be processed by which expert. Second, a dispatching stage is responsible for routing each token to its respective mapped experts' input buffers. Note, experts could reside on different devices, requiring the need for an all-to-all to exchange tokens between devices. Third, each expert independently processes all the tokens mapped to it. Fourth, a combine stage re-routes the tokens back to their original device via a second all-to-all, combining all the expert outputs and reordering the tokens to match the order of the original tokens input to the layer. Fig. 1(a) illustrates the MoE architecture.

Expert-Specialized Mixture-of-Experts. The MoE architecture design paradigm has recently evolved. Prior to DeepSeek-MoE [10], state-of-the-art MoEs like Mixtral-MoE use a small number of experts (e.g., 8), each of which is similar to the FFN layers in corresponding dense models with a small top-k gating value (e.g., k is typically 1 or 2). However, such models are shown to lack expert specialization [17]. To overcome the issue, DeepSeek-MoE introduces both fine-grained experts and large top-k routing to increase expert specialization (Fig. 1(b)). Instead of a few large experts, the model uses a much higher number of smaller experts and activates more of them for reach token. Concretely, each expert's hidden dimension is reduced to a fraction (e.g., $1/m$, where $m \propto k$) of the size used in a standard MoE's FFN. Meanwhile, the number of experts increases roughly m-fold. For example, if a standard MoE has an FFN dimension of 4096 with 8 experts (activating 2 per token), DeepSeek-MoE splits this into $m = 8$ fine-grained experts per original expert dimension, yielding 64 experts total and activating 16 per token. This keeps the total parameters and per-token computation roughly the same as before but dramatically expands the combination of experts a token can see from only 28 possible expert-pair combinations in the original example to nearly 4.89×10^{14} combinations. DeepSeek-MoE models employ on the order of hundreds of experts per MoE layer, e.g., DeepSeek-v3 uses 256 experts in each layer, with 8 experts activated for every token, which exhibits far more expressive power than standard MoEs with the same parameter budget.

Existing MoE Training Frameworks. Training MoE models at scale introduces a number of system-level challenges. Early systems, such as GShard, introduce Expert Parallelism (EP) [23], which enables efficient MoE scaling across devices by distributing experts across GPUs. Subsequently, several large-scale MoE training frameworks were introduced [9, 14, 15, 18, 30, 33]. DeepSpeed-MoE combines ZeRO-style Data Parallelism (ZeRO-DP) with EP, offering more memory-efficient training for large-scale MoE models. A more recent extension of DeepSpeed-MoE, DeepSpeed-TED [33], introduces three-dimensional parallelism by combining DP, EP, and Tensor-slicing Parallelism (TP) to scale MoEs further. However, they are designed for low top-k values and coarse-grained experts. Tutel [16] proposes an adaptive DP and TP strategy, optimizing memory and compute usage by dynamically switching between data and tensor parallelism depending on the load distributed amongst experts. Recently, Megablocks [14] introduces sparse primitives and a no-token dropping scheme to process MoE layers, representing everything as block-sparse matrix multiplications. However, in doing so, their kernel requires padding the token buffer to multiples of a preset size, incurring serious zero-paddings on the emerging MoE workload.

3 Challenges and Opportunities

Expert-specialized MoEs represent a significant advancement in LLMs. However, training these emerging MoE architectures poses significant challenges for existing off-the-shelf MoE training solutions, especially on HPC platforms.

3.1 Lack of Efficient Cross-Platform Kernels for Scaling Expert-Specialized MoEs

Existing MoE training frameworks often rely on a dense and static tensor layout for MoE gating and dispatching, which becomes inefficient when applied to expert-specialized MoEs. For example, MoE training frameworks (e.g., GShard [23], Fairseq [29] and DeepSpeed-MoE [30]) often implement each stage of the MoE pipeline via fast batched matrix multiplication (matmul) primitives. However, these primitives place a constraint: requiring the same number of tokens routed to each expert, which does not hold during training. To handle the dynamic assignment of tokens to experts, these frameworks introduce a fixed expert capacity C and pad unused slots with zeros when fewer than C tokens are routed to an expert. Conversely, tokens are dropped when capacity is exceeded. During the gating stage, a dispatching mask, `dispatch_mask` of size `[S, E, C]` is constructed. The entry `dispatch_mask[t, e, c]` is either 1 or 0 indicating if the t^{th} token is routed to the c^{th} position in expert `e`'s buffer. A token-dropping mask is applied over `dispatch_mask` to additionally drop tokens.

Each of the dispatch, MLP and combine stages leverage matmuls on input token-buffers to process tokens. During the dispatch stage, each worker uses an `einsum` operation on the dispatching mask

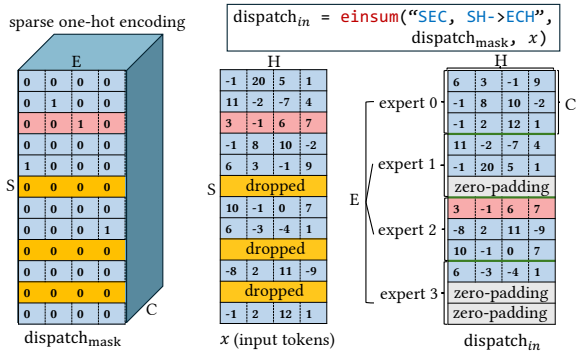


Figure 2: Conventional gating and dispatching logic with zero-padded and large intermediate memory cost.

and input tokens to correctly place each token into its respective experts buffers. The expert buffers are `[E, C, H]`-sized. If less than C tokens are placed in an expert's buffer, the unused slots are zero-padded. Fig. 2 illustrates the dispatch process and how zero-padding is introduced into an expert's input-buffer at this stage. Next, an even `alltoall` communication exchanges token buffers across devices, correctly routing each token to the devices its experts reside on. Each expert then operates on its input token-buffers in parallel. Finally, another `alltoall` re-exchanges the tokens to their original device to generate the final output of the layer. Importantly, the zero-padding introduced in the dispatch stage is retained across each `alltoall` and expert compute. This increases both the communication volume and activation memory of MoE training.

In expert-specialized MoEs, where hundreds of fine-grained experts are activated per MoE layer and top-k routing is large, the above zero-padded pipeline becomes a major memory bottleneck. In our experiments with DeepSeek-style MoEs, we observe that the dispatch mask and intermediate buffers consume over 70% of the total activation memory when training with DeepSpeed-MoE [30] using 1.25× average perceived tokens per-expert as the expert capacity. This not only increases the pressure on GPU memory but also the communication volume of `alltoall` calls dependent on these buffers. While it is possible to build sparse CUDA kernels to address this issue [16], CUDA kernels are tightly coupled to NVIDIA's CUDA backend and cannot be easily ported to other platforms. The lack of cross-platform support becomes a critical limitation on different HPC systems. As a result, the current training pipeline either falls back to inefficient PyTorch-based implementations or requires costly kernel re-engineering (e.g., ROCm-based kernels) for the particular hardware target in question, which is error-prone. As MoE architectures grow more complex, the need for portable, hardware-agnostic MoE kernels becomes very essential.

Takeaway-1: Existing MoEs rely on dense, CUDA-specific implementations that are inefficient for expert-specialized MoEs and difficult to port to non-NVIDIA platforms. This lack of cross-platform support leads to inflated memory usage and degraded performance of MoE training on heterogeneous HPC platforms.

3.2 Memory Bottleneck Shift in Expert-Specialized MoEs

We analyze the unique memory behavior of emerging expert-specialized MoE architectures by comparing them with size-equivalent conventional MoEs, where we keep the total parameters and per-token activated parameters the same. We refer to these as M_{conv} (conventional MoE) and M_{spec} (expert-specialized MoE). Table 1 summarizes the key differences. We use E to denote the number of experts, H for the model dimension, and H_{FFN} for the intermediate hidden dimension in the expert FFN layers. We use m to denote *fine-grained factor*, which indicates how many fine-grained experts in M_{spec} together replace a single expert in M_{conv}. For example, in most conventional MoEs [13, 17, 30], m = 1; in DeepSeek [11], m=8 because expert FFNs have a ∼8× smaller hidden dimension and each token activates k=8 experts.

Model	E	H	H_{FFN}	k	Param	Activated Param
base	-	h	h'	-	$2h'h$	$2h'h$
M_{conv}	e	h	h'	1	$2eh'h$	$2h'h$
M_{spec}	$e \cdot m$	h	h'/m	m	$2eh'h$	$2h'h$

Table 1: The model configurations of size-equivalent conventional MoE M_{conv} and expert-specialized MoE M_{spec}.

Training memory consists of model states and activations. Since M_{conv} and M_{spec} are size-equivalent, their model state size is identical. However, their activations behave very differently. Each MoE layer often instantiates four key activations during training:

- $A_{\texttt{dispatch}}$: dispatched input to experts;
- $A^0_{\texttt{interm}}$ and $A^1_{\texttt{interm}}$: intermediate activations between expert FFN sub-layers;
- $A_{\texttt{combine}}$: expert outputs before combining.

All four tensors scale with batch size b, sequence length s, and routing factor k. Among them, $A_{\texttt{dispatch}}$ and $A_{\texttt{combine}}$ scale with the model dimension H, while $A^0_{\texttt{interm}}$ and $A^1_{\texttt{interm}}$ scale with the expert hidden dimension H_{FFN}. However, given that H_{FFN} in $A^0_{\texttt{interm}}$ and $A^1_{\texttt{interm}}$ shrink by 1/m in M_{spec} and $k \propto m$, $A^0_{\texttt{interm}}$ and $A^0_{\texttt{interm}}$ remain constant across M_{conv} and M_{spec}. Instead, $A_{\texttt{dispatch}}$ and $A_{\texttt{combine}}$ grow linearly with m. Therefore, in expert-specialized MoEs, activation memory is primarily dominated by the dispatch and combine stages. Table 2 summarizes this. This finding is interesting because prior work often assume that the intermediate FFN activation is large (e.g., 4H) for M_{conv}. Our analysis reveals that this assumption no longer holds true for expert-specialized MoEs.

Activation	$A_{\texttt{dispatch}}$	$A_{\texttt{combine}}$	$A^0_{\texttt{interm}}$	$A^1_{\texttt{interm}}$
Tensor Size	$kBSH$	$kBSH$	$kBSH_{FFN}$	$kBSH_{FFN}$
M_{conv}	bsh	bsh	bsh'	bsh'
M_{spec}	$mbsh$	$mbsh$	bsh'	bsh'

Table 2: The activation size of equivalent conventional MoE model M_{conv} and expert-specialized MoE model M_{spec}.

We further validate this observation in Fig. 3, which compares the per-GPU memory consumption of one M_{conv} and M_{spec} layer when trained with ZeRO-1 DP and EP on 256 GPUs, using an EP

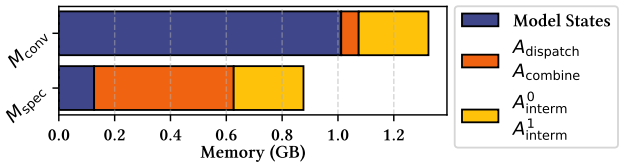


Figure 3: The MoE layer memory distribution of M_{conv} and M_{spec} created by a 6.7B base model [6] with $e = 16$ and $m = 8$.

size (i.e., expert-parallel group size) equal to the number of experts. The results show a clear shift in memory bottleneck.

> **Takeaway-2:** As expert-specialized MoEs increase the number of routed experts and shrink expert hidden dimension, per-device memory bottlenecks shift from model parameters to activations, particularly in the dispatch and combine stages.

3.3 Expensive All-to-All on HPC Platform with Heterogeneous and Hierarchical Network

Most prior MoE training systems were designed for clusters composed of NVIDIA DGX nodes [16, 30], where nodes are connected via high-bandwidth, low-latency InfiniBand. These clusters exhibit relatively balanced GPU-to-GPU inter/intra-node communication bandwidths, with intra-node bandwidths only 3× faster than inter-node bandwidths. As a result, existing MoE systems often take advantage of this balanced network and treat all GPUs in a cluster equivalently. For example, in DeepSpeed-MoE [30], each collective involves all GPUs within an expert-parallel group, regardless of physical placement.

However, many HPC platforms differ from DGX-style clusters. For example, Frontier [5] adopts a Dragonfly topology, where GPUs within a node are connected via Infinity Fabric (up to 200 GB/s) while inter-node communication happens via Slingshot (25 GB/s). This introduces a significant bandwidth asymmetry. In such hierarchical interconnects, network-aware communication is essential. Unfortunately, existing MoE systems do not exploit this hierarchy and instead route tokens indiscriminately, often leading to sub-optimal bandwidth utilization.

This problem is exacerbated in expert-specialized MoEs, which rely on large top-k routing where each token is sent to a large number of experts. If multiple selected experts reside on the same node, existing systems send multiple copies of the same token activation across inter-node links, one for each destination expert, even though only one copy is actually needed (§ 4.2). To quantify this redundancy, we evaluate a DeepSeek-style configuration (256 experts, k=8 routing) using DeepSpeed-MoE. Fig. 4 shows that the redundancy rate ranges can be up to 75.1%, depending on the EP size. This leads to redundant inter-node communication.

> **Takeaway-3:** Expert-specialized MoEs increase the number of routed expert per token, leading to significant duplication in communication. On HPC platforms with hierarchical and heterogeneous networks, this results in inefficient use of inter-node bandwidth and causes communication to become a major training efficiency bottleneck as the expert granularity increases.

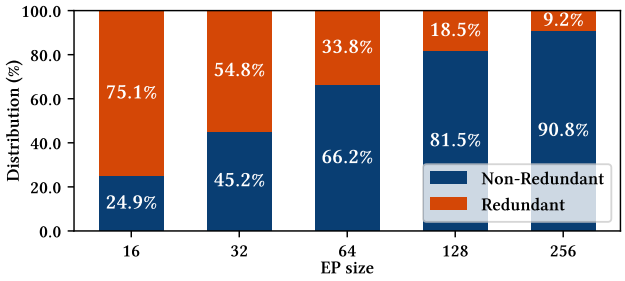


Figure 4: Redundancy rate of all dispatched tokens.

4 X-MoE Design

To address the training inefficiencies of emerging MoE architectures on non-NVIDIA platforms, X-MoE introduces a set of system optimization techniques to address the unique challenges posed by fine-grained experts with large top-k routing on hierarchical HPC platforms. Fig. 5 provides an overview of X-MoE. First, we propose a new sparse data layout PFT and rework the MoE pipeline to eliminate zero-padding across different MoE stages via padding-free sparse MoE training with cross-platform kernels (§ 4.1). Second, we reduce communication redundancy by leveraging topological awareness of HPC systems via a hierarchical two-stage redundancy-bypassing dispatch algorithm (§ 4.2). Third, X-MoE incorporates a hybrid prallelism strategy that enables sequence-sharding in MoE blocks to address the activation memory bottleneck, with topology-aware planning and device-mapping. Together, these optimizations form an integrated and cross-platform training system that scales emerging MoEs on large scale HPC clusters. The following sections provide an in-depth description of each design.

4.1 Padding-Free Sparse MoE Training with Cross-Platform Kernels

We introduce the first truly padding-free MoE training pipeline. First, we design a novel sparse data-structure: PFT (**P**adding-**F**ree **T**oken buffers). The PFT is designed to eliminate zero padding through the MoE computation and communication stages, including the dispatch, MLP, and combine stages (§ 4.1.1). Instead of allocating fixed-capacity expert buffers padded with zeroed-vectors, PFT stores only valid routed tokens. However, in introducing the PFT to MoE training, each of the dispatch, MLP, and combine stages needs to be modified to operate on the PFT. We also detail our modifications to each stage (§ 4.1.1). Second, to efficiently implement the PFT-based pipeline, we design a suite of Triton-based kernels to handle the corresponding sparse and irregular workloads (§ 4.1.2). These kernels are designed to be hardware-agnostic, support coalesced memory accesses and avoid vendor-specific constraints like CUDA-only fused kernels. As a result, our padding-free MoE training pipeline improves memory efficiency and reduces communication volume, which serve as key enablers of scalable training of expert-specialized MoEs on diverse hardware.

4.1.1 Padding-Free Token Storage and Pipeline To eliminate the inefficiencies introduced by zero-padding in existing MoE pipelines, we introduce the PFT data-structure. Unlike standard expert input buffers that reserve fixed-capacity slots per expert, PFT consists of a token-buffer, `x`, which stores only the routed tokens, along with

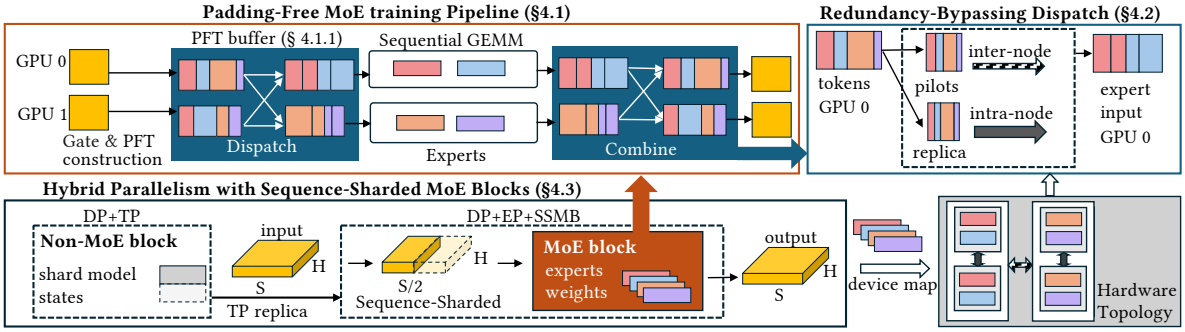


Figure 5: Overview of X-MoE. At a high level, X-MoE enables efficient and scalable training for expert-specialized MoEs through a set of targeted system-level optimizations, including a fully padding-free MoE training pipeline with cross-platform kernels (§ 4.1), redundancy-bypassing dispatch (§ 4.2), and hybrid parallelism with sequence-sharded MoE blocks (§ 4.3).

expert routing information arrays (ERI-arrays) that track how each token should be processed. The ERI-arrays consists of the following data: (1) array `token_ids` (a `[B]`-sized array; `B` is the number of routed tokens in `x`) where `ti = token_ids[i]` is an index that maps the ti^{th} input-token to the i^{th} position in the dispatch matrix (see figure Fig. 6 for an example), (2) array `expert_ids` (a `[B]`-sized array) where `expert_ids[i]` represents the expert that `x[i]` is routed to, (3) array `tokens_per_expert` (a `[E]`-sized array; `E` is the expert count) where `tokens_per_expert[i]` represents the number of tokens in `x` routed to expert `i`, (4) array `combine_weights` (a `[B]`-sized array) where `combine_weights[i]` represents the value that $\texttt{combine}_{\texttt{in}}\texttt{[i]}$ (an intermediate matrix assembled after the last `alltoall`, described later in § 4.1.1) is scaled by in the combine phase. We first show how PFT is constructed and then demonstrate how this representation allows each MoE stage to operate without any zero padding. Fig. 6 depicts the PFT structure with ERI-arrays and how ERI-arrays drive token dispatching.

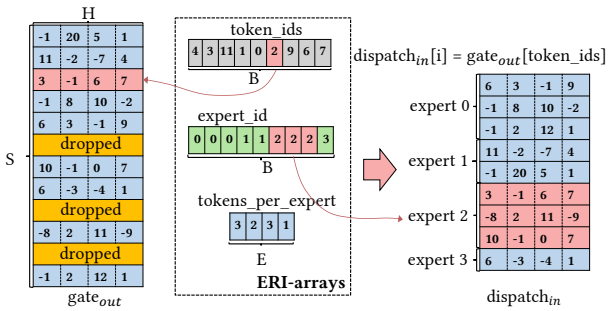


Figure 6: PFT with sparse structure and ERI-arrays.

PFT Construction. PFT is constructed after the MoE gating function and before token dispatching (we describe how the gating, dispatch, MLP and combine stages are modified later). Listing 1 illustrates the pseudo code for PFT construction. It takes as input the outputs of the gating function: (1) the `top_experts` array, a `[S, E]`-sized array that contains the token to expert mapping and (2) corresponding `combine_weights` array, a `[S, E]`-sized array used in the combine stage containing each token's probability score that reflects the confidence of the gating function. It also takes as input the `max_token_count` variable, indicating the expert-capacity. The PFT construction routine returns a PFT whose ERI-arrays are correctly instantiated.

The PFT construction routine proceeds in two stages. In the first stage, we flatten and sort the incoming `top_experts` array (lines 20-21), which contains the token to expert assignments generated by the gating function. In the second stage, we determine which tokens are dropped (lines 24-33); using this information, we construct the ERI-arrays by pruning out the dropped tokens from the unfiltered `token_ids`, `expert_ids`, and `combine_weights` (lines 34-36).

Padding-free Gating, Dispatch, MLP and Combine. Listing 1 (lines 67-72) illustrate our padding-free pipeline where the modified dispatch, MLP and combine stages operate on the PFT. First, during gating, we transform the input tokens to logits and select the top-k experts per token, returning their respective expert indices (`top_expert`) and probability confidence scores of the assignment (`combine_weights`) (lines 6-8). Second, we construct the PFT structure using these data. Third, during dispatch, we consume the pft and $\texttt{gate}_{\texttt{out}}$ tokens and route tokens to the correct worker. This occurs by: (1) reordering the tokens locally using a custom gather-kernel (described in § 4.1.2) producing the $\texttt{dispatch}_{\texttt{in}}$ buffer, (2) exchanging the tokens between devices via an uneven `alltoall`, routing each token to the correct device its expert resides on (lines 43-47) producing the $\texttt{dispatch}_{\texttt{out}}$ buffer. No zero-padding is communicated in this stage. Fourth, the MLP layer processes tokens; we launch a custom sequential-GeMM (described in § 4.1.2) to implement each MLP layer enabling different token-counts to be multiplied by the MLP weights of different experts without the need for zero-padding (lines 50-53). Fifth, during combine, tokens are re-routed back to their original device and a custom scatter kernel (described in § 4.1.2) locally reorders the inbound tokens to their original position in the sequence, multiplying each token with its respective value in `combine_weights` (lines 56-58).

4.1.2 *Highly-Optimized Cross-Platform Sparse and Irregular Kernels*

The PFT format helps improve the memory-efficiency of emerging MoE training by eliminating the need for any zero-padding in the dispatch, MLP and combine stages. However, certain operators in this modified pipeline, such as the gather, scatter and sequential GeMM, introduce sparse and irregular access patterns to the PFT ERI-arrays, which can be expensive to implement in Pytorch and requires specialized kernels for efficiency. To address these issues, we introduce Triton-based gather, scatter as well as (non Triton-based) sequential GeMM implementations that are high-performance and platform agnostic. The gather and scatter kernels are responsible

```
def gating(k,tokens):
  """
  k: int.
  tokens: input tokens to MoE-layer [S, H]-sized.
  """
  logits = softmax(FFN(tokens), axis=-1)
  combine_weights, top_experts = topk(logits)
  return top_expert, combine_weights, tokens

def PFT_construction(max_token_count,top_experts,
    combine_weights):
  """
  top_experts: token to expert assignments [S, E]-sized
  combine_weights: gating probabilities [S, E]-sized
  max_token_count: maximum tokens for a single expert
  Returns generated PFT
  """
  E = get_expert_count()
  ## Step 1: Generate the expert & token ids ##
  # shape (lines 20-21): [S*K]
  flat_top_experts = flatten(top_experts)
  expert_ids, token_ids = sort(flat_top_experts)
  ## Step 2: Identify filter dropped-tokens ##
  # shape (lines 24-26): [S*K]
  flat_combine_weights = flatten(combine_weights)
  sorted_indices = argsort(flat_combine_weights)
  sorted_top_experts = flat_top_experts[sorted_indices]
  # shape (lines 28-30): [S*K, E]
  one_hot_enc=one_hot(sorted_top_experts,num_classes=E)
  rank_in_expert = cumsum(one_hot_enc, axis=0)
  weight_mask = rank_in_expert <= max_token_count
  # shape (lines 32-36): [B], token-count post dropping
  filtered_indices = sorted_indices[weight_mask]
  retained_token_ids = isin(flat_top_experts,
    filtered_indices)
  token_ids = token_ids[retained_token_ids]
  expert_ids = expert_ids[retained_token_ids]
  combine_weights = combine_weights[retained_token_ids]
  # shape: [E]
  tokens_per_expert = histogram(expert_ids, bins=E)
  # Returns a PFT with generated ERI-arrays
  return PFT(token_ids,expert_ids,tokens_per_expert,
    combine_weights)

def dispatch(pft, gate_out):
  dispatch_in = gather_kernel(gate_out,pft.token_ids,pft.
    expert_ids)
  pft.tokens_per_expert=alltoall(pft.tokens_per_expert)
  dispatch_out = alltoallv(dispatch_in, pft.
    tokens_per_expert)
  pft.x = dispatch_out
  return pft

def mlp(pft, w1, w2):
  inter_activ = sequential_gemm(pft.x,w1)
  mlp_out = sequential_gemm(inter_activ,w2)
  pft.x = mlp_out
  return pft

def combine(pft):
  combine_in = alltoallv(pft.x,pft.tokens_per_expert)
  combine_out = scatter_kernel(combine_in,pft.token_ids,
    pft.expert_ids,pft.combine_weights)
  return combine_out

def call(tokens, k, max_token_count, w1, w2):
  """
  tokens: tokens input to the MoE layer, [S, H]-sized
  k: topk value, int.
  max_token_count: expert capacity, int.
  w1 & w2: weights of first and second layer of  MLP.
  """
  top_expert,combine_weights,gate_out = gating(k,tokens)
  pft = PFT_construction(max_token_count,top_expert,
    combine_weights)
  pft = dispatch(pft,gate_out)
  pft = mlp(pft,w1,w2)
  pft = combine(pft)
  return pft.x
```

Listing 1: Padding-free MoE-layer

for computing: $\texttt{dispatch}_{\texttt{in}}[i,:] = \texttt{gate}_{\texttt{out}}[\textit{token\_ids}[i],:]$ and $\texttt{combine}_{\texttt{in}}\texttt{[token\_ids[i], :]} = \texttt{mlp}_{\texttt{out}}\texttt{[i, :]} \times \texttt{combine\_weights[token\_ids[i], :]}$, respectively. However, the irregular memory access patterns in reading and writing to tensors results in uncoalesced memory requests and poses a unique performance challenge. We circumvent this by scheduling a single thread-block to operate (read and write) on one vector, assigning contiguous threads to operate on the model-hidden dimension (outer-dimensions of the $\texttt{gate}_{\texttt{out}}$ and $\texttt{combine}_{\texttt{in}}$ tensors). On the other hand, our sequential GeMM operates on the $\texttt{dispatch}_{\texttt{out}}$ matrix. It extracts the correct tokens each expert is assigned to in $\texttt{dispatch}_{\texttt{out}}$ with a python for-loop launching $\texttt{E}_{\texttt{local}}$ (number of experts assigned to the device) GeMMs.

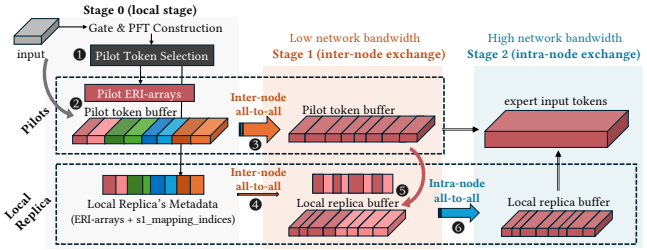


Figure 7: Hierarchical Redundancy-Bypassing Dispatch: Multi-stage token routing across inter- and intra-node networks to reduce communication duplication.

4.2 Redundancy-Bypassing Dispatch

We propose Hierarchical Redundancy-Bypassing Dispatch (RBD) to eliminate redundant inter-node communication by introducing a multi-stage dispatching process with two groups of tokens: *Pilot tokens*, which are the minimal set of distinct tokens that must be communicated across nodes; and *local replica*, which are local duplicates of pilot tokens routed to additional experts on the same destination node. Instead of sending all token data through one `alltoall`, RBD only sends pilot tokens through inter-node communication and propagates local replica using fast intra-node connects. We now illustrate RBD's multi-stage process using Fig. 7.

***Stage 0 (S0)*: Pilot Selection and Instantiation.** The first step of RBD is *pilot tokens selection* within x, which is generated through PFT in § 4.1. Based on `token_ids` and `expert_ids`, RBD extracts the node destination information for each token. Then for each token's k destinations, RBD identifies the group of experts that share the same destination node. Among tokens with the same source and destination node, RBD randomly selects one as the pilot token and marks the rest as local replica. This randomized strategy helps avoid a biased distribution and creates a balanced workload for `alltoall` communication. For example, always routing tokens to the smallest expert ID within a node will significantly increase the `alltoall` latency.

Meanwhile, we create separate ERI-arrays for pilot tokens and local replicas, respectively. Each of them contains the routing information for those tokens. This process is represented by ❶ in Fig. 7. In addition, we construct a mapping array `s1_mapping_indices` (used in Stage 1 for local replica reconstruction), where each local replica token records the index of its corresponding pilot token. To ensure the correctness of this mapping index before and after the uneven `alltoall` exchange (❹), we use the relative index starting from 0 for each target expert. This is allowed because the pilot ERI-arrays is sorted by expert IDs. We re-encode it to the absolute index after the `alltoall` exchange. Finally, we instantiate the pilot token buffer (❷) using a Triton gather kernel. The local replica tokens are

not instantiated yet. They are reconstructed from their associated pilot tokens after the pilot tokens arrive their destination.

***Stage 1 (S1)*: Inter-Node Exchange (Pilot Only) and Local Replica Reconstruction.** In S1, RBD sends only pilot tokens across nodes using an uneven `alltoall` (❸). This is the only stage that uses inter-node bandwidth. Additionally, RBD also sends local replica tokens' metadata (ERI-arrays and `s1_mapping_indices`) (❹), alongside their corresponding pilot tokens. This is lightweight given that metadata has small message size. Once the pilot tokens arrive at their destination node, local replica tokens are reconstructed by copying data from pilot tokens to a local exchange buffer based on `s1_mapping_indices` (❺). Note that the local exchange buffer serves as the input of the intra-node `alltoall`, RBD ensures token data is contiguous and ordered by destination (e.g., the ascending order of expert IDs).

***Stage 2 (S2)*: Intra-Node Exchange (Local Replica Only) and Expert Input Reconstruction.** The newly reconstructed local replica tokens are exchanged among GPUs within the same node using a fast intra-node uneven `alltoall` ❻, which helps to save expensive inter-node traffic. After pilot tokens and local replica tokens all arrive at their target GPUs, RBD reconstructs the each expert's local input by merging the two groups and correctly orders them based on their expert indices.

The combine stage reverses the above described RBD process. Specifically, local replica tokens are first gathered via intra-node communication, followed by pilot tokens through inter-node transfer. To ensure the correctness of combining weight scaling on expert outputs, we exchange the original `combine_weights` ERI-array for all tokens in advance through a small inter-node `alltoall`, along with ❹. During combine, the weight scaling is performed in *stage 1*, before merging local replica tokens into pilot tokens. Finally, the full results are reconstructed from the pilot tokens on the original device using the ERI-arrays preserved during dispatch.

4.3 Hybrid Parallelism with Sequence-Sharded MoE Blocks

Training MoEs at scale requires hybrid parallelism that carefully balances memory, compute, and communication. A common approach is to apply tensor parallelism (TP) to dense blocks (e.g., attention, non-MoE MLPs) as in Megatron-LM [25] and switch to expert-parallelism (EP) for MoE blocks, sharding expert weights across devices. This TP + EP combination, used in systems such as DeepSpeed-TED [33], allows scaling conventional MoE parameters across large clusters. However, the naive transition from TP to EP fails to address the key bottleneck in expert-specialized MoEs: the activation memory, especially for $A_{\texttt{dispatch}}$ and $A_{\texttt{combine}}$. As described in § 3.2, these tensors scale linearly with sequence length s, routing factor k, hidden dimension h, and fine-grained factor m.

Tensor parallelism works by duplicating input tokens across all TP ranks and computing partial results, which are later reduced via all-reduce. In the context of MoE training, this means that each TP worker holds a full copy of the input sequence, and even if we switch to EP within the MoE block, each EP worker begins the MoE computation with the same *duplicated activations*. Consequently, the heavy activations ($A_{\texttt{dispatch}}$ and $A_{\texttt{combine}}$ as described in § 3.2)

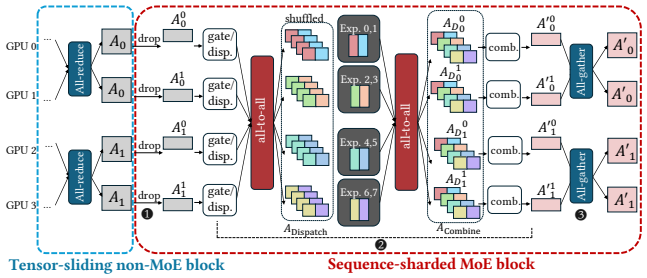


Figure 8: Illustration of X-MoE's hybrid parallelism with sequence-sharded MoE blocks.

are not reduced at all, as they are still duplicated across all EP workers that originate from TP ranks, which leads to poor scaling.

To address this challenge, we propose a new hybrid parallelism strategy that combines tensor-slicing parallelism with sequence-sharded execution for MoE Block (SSMB). This strategy is motivated by a key insight: all operations in an MoE block (gating, dispatch, expert FNNs, and combine) are applied token-wise and do not require inter-token dependencies. This allows us to shard the input sequence of the MoE block across EP ranks, so each rank only retains and processes a segment of the sequence and later recover the full sequence using all-gather, reintroducing the duplicated inputs expected by the next TP block. This strategy reduces the activation footprint of $A_{\texttt{dispatch}}$ and $A_{\texttt{combine}}$ by a factor of the TP group size, while preserving compatibility with standard MoE routing and communication.

Fig. 8 illustrates how SSMB works in practice. In this setup, we use TP=2 and DP=2 (TP and DP parallel-group size) for the dense (non-MoE) block, and EP=4 for the MoE blocks. In the TP + DP phase, each TP worker holds a full copy of the input sequence: device 0 and 1 each have a copy of sequence A_0, while device 2 and 3 have A_1. In existing MoE training, duplicated activations like $A_{\texttt{dispatch}}$ and $A_{\texttt{combine}}$ would be store on both devices, increasing memory cost. Instead, SSMB drops a fraction of the tokens on each device (❶), partitioning the sequence across TP ranks (e.g., A_0^0 and A_0^1). After entering the MoE block, SSMB reassigns each TP+DP worker to act as an EP rank and performs MoE-Gating, dispatch, expert FNNs, and combine on the partitioned tokens (❷), using the padding-free pipeline introduced in § 4.1. After the combine op, SSMB issues an all-gather (❸) to reconstruct the full output sequence (e.g., A'_0) across all EP ranks, effectively restoring the replicated data layout for the next TP-based non-MoE block.

In the backward pass, SSMB follows a reversed sequence of operations. Upon entering the MoE block, it first drops the gradients corresponding to the partial sequences retained during forward. It then performs expert-specific gradient computation and `alltoall` communications, mirroring the forward process. Finally, SSMB uses an all-gather operation to reconstruct the full input gradient across TP ranks, allowing the propagation to continue in the TP phase.

Can existing parallel strategies handle the shifted memory bottleneck? The careful reader may think of an alternative approach that uses TP + EP for MoE blocks, as opposed to EP with sequence sharding. After all, these schemes also shard model states across devices and reduce memory. However, in expert-specialized MoEs, experts already have small intermediate dimensions, making TP's benefits marginal. Moreover, neither TP nor ZeRO-style DP reduce

the expensive activations like $A_{\texttt{dispatch}}$ and $A_{\texttt{combine}}$. We compare the overhead and gain of using SSMB with tensor-expert-data (TED) parallelism by calculating both model states and activation memory saving of each approach. In short, the benefit of SSMB over TED depends on the ratio: $r = \frac{k}{H_{FFN}}$. Under the usage of ZeRO-1 DP, when this ratio satisfies: $r > \frac{2}{c \cdot S}$, SSMB offers more memory savings than TED. For expert-specialized MoEs with fine-grained factor m, we have $H_{FFN} \propto \frac{1}{m}$ and $k \propto m$. Thus, generally speaking, under identical sequence length choice, the more fine-grained the MoE model is, the more benefits SSMB provides over TED.

Why not activation checkpointing? Another approach to reducing activation memory is activation checkpointing [8], which trades memory for recomputation. However, in MoE training with expert parallelism, $A_{\texttt{dispatch}}$ and $A_{\texttt{combine}}$ are outputs of `alltoall` communication during token routing. Applying checkpointing to these tensors would require `alltoall` communication during the backward pass, incurring expensive communication overhead in addition to the recomputation overhead.

Why not use pipeline parallelism (PP)? While PP is effective for reducing memory by splitting the model across devices, it requires significant code refactoring and careful scheduling to balance pipeline stages, especially with sparse MoE layers. In contrast, our solution requires minimal code changes. We leave the integration with PP as future work.

5 Evaluation

In this section, we evaluate X-MoE in comparison with state-of-the-art large-scale MoE training approaches, demonstrating that it achieves significantly improved training efficiency and scalability for emerging MoEs. We also show the impact of different technologies within X-MoE on performance.

5.1 Evaluation Methodology

Hardware. We conduct evaluation on the *Frontier* supercomputer [5]. Each cluster node is equipped with 4×AMD MI250X GPUs with dual Graphics Compute Dies (GCDs) and one EPYC CPU. A GCD is viewed as an effective GPU. The 2 GCDs on the same MI250X are connected with Infinity Fabric with a peak bandwidth of 200 GB/s. The GCDs on different MI250X are connected with Infinity Fabric where the peak bandwidth ranges from 50-100 GB/s. The Frontier nodes are connected with four Slingshot 25 GB/s NICs. We use up to 128 nodes (1024 MI250X GCDs) for experiments.

Evaluation setup. We implement X-MoE in DeepSpeed [31], a widely used open-source DL training library. We include the implementation and environment details in Appendix. If not specified, we choose the maximum micro-batch size of power of 2 under the memory limitation and a global batch size of 1024. We choose the capacity factor $c = 1.25$ for all experiments, as suggested by [23].

5.2 Main Results

We first demonstrate that X-MoE scales effectively across a wide range of expert-specialized MoE models. We use model configurations from DeepSeek-MoE [10], DeepSeek-v2 [24], and DeepSeek-v3 [11], as shown in Table 3. We compare X-MoE against three large-scale MoE training frameworks: DeepSpeed-MoE [30], DeepSpeed-TED [33], and Tutel [16] as baselines. For DeepSpeed-MoE and Tutel, we sweep EP size in {32/64/128/256} and ZeRO stages 1/2. For DeepSpeed-TED, we additionally sweep TP in {1, 2, 4, 8} and choose the best performing configuration.

Models	Small	Medium	Large	Super
seq. length	2048	4096	4096	4096
H_{model}	2048	5120	7168	7168
H_{FFN}	1408	1536	2048	2560
num. experts	64	128	256	256
top-k	6	6	8	8
num. layers	28	28	28	61
Param.	10.1 B	55.2 B	201.4 B	545.4 B
Activated Param.	1.3 B	5.2 B	11.5 B	28.7 B

Table 3: The model configs used for evaluation.

Trainability and throughput. We evaluate X-MoE on Small (10.1B), Medium (55.2B), and Large (201B) model configurations using 256 GPUs. As shown in Fig. 9, while existing systems such as DeepSpeed-MoE, DeepSpeed-TED, and Tutel run out of memory on medium and large models, X-MoE successfully enables their training, effectively changing the status from non-trainable to trainable under the same hardware budget. When multiple systems can train a model, e.g., on the *Medium* model, X-MoE achieves higher throughput, with 5.15x and 1.42x speedup over DeepSpeed-TED and Tutel respectively. X-MoE achieves these results through a set of targeted system-level optimizations. Its padding-free training pipeline eliminates zero-padding overhead in both memory and communication. RBD reduces communication redundancy in high top-k routing scenarios by minimizing cross-node token duplication. SSMB effectively mitigates the shifted memory bottleneck. Together, these innovations enable efficient and scalable training of emerging expert-specialized MoEs.

Pushing the model scale limit. X-MoE further enables the *Super* 545B model on 1024 GPUs, achieving an aggregated throughput of 10.44 PetaFLOPs while all prior systems fail due to OOM errors. At this scale, training becomes sensitive to system dynamics beyond memory and communication volume optimizations. On Frontier, we observe that scaling beyond 256 GPUs results in significantly higher `alltoall` latencies (> 10× higher than average), likely due to increased cross-rack communication and network congestion from concurrently running jobs on the shared cluster. Despite this, X-MoE successfully sustains large-scale MoE training across racks, demonstrating its robustness and extending the boundary of what is trainable on today's HPC clusters.

5.3 Scalability Evaluation

We evaluate both weak and strong scaling to demonstrate that X-MoE not only enables training of expert-specialized MoEs but also scales efficiently with increasing compute resources. We compare against only Tutel, as it is the best performing baseline as shown in Fig. 9.

Weak scaling. To evaluate weak scaling behavior, we train the 10.1B *Small* model from 16 to 256 GPUs, proportionally increasing the global batch size from 256 to 4096. We use EP=8 and scale out via ZeRO-DP. Our results are in figure Fig. 10(a). The results show that X-MoE consistently maintains higher TFLOPs compared to Tutel

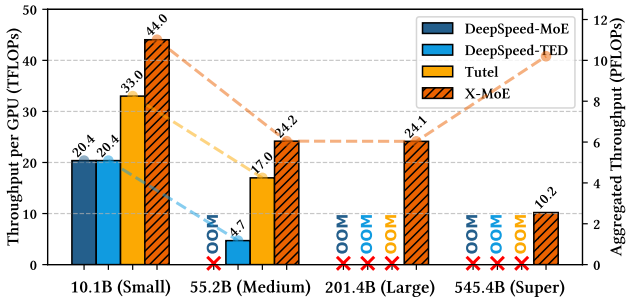


Figure 9: Results on training Small, Medium, and Large models on 256 GPUs; training Super model on 1024 GPUs. The dashed lines show the aggregated throughput.

with a comparatively smaller drop in throughput as the number of GPUs increases.

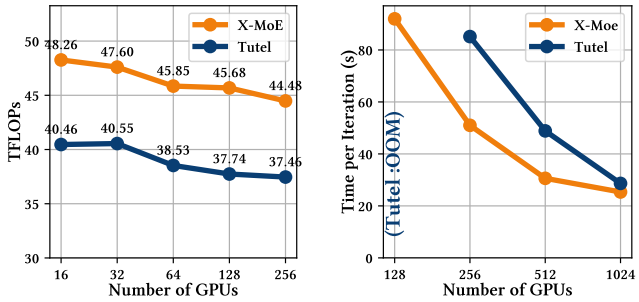


Figure 10: Scalability results. (a) Weak scaling results: Training the 10.1B MoE on 16–256 GPUs with increasing batch size. (b) Strong scaling result: Training the 55.2B MoE across 128, 256, 512, 1024 GPUs while fixing the batch size.

Strong scaling. We evaluate strong scaling using the 55.2B *Medium* model on 128, 256, 512, and 1024 GPUs, keeping the global batch size fixed at 2048. This setup tests how well the system reduces iteration time as more GPUs are used. Since DeepSpeed-MoE fails to run due to OOM errors, we compare X-MoE (EP=64) with Tutel (EP=128). Fig. 10b shows that Tutel cannot run on 128 GPUs even with EP=128, while X-MoE scales effectively and achieves lower iteration time as GPU count grows. At 1024 GPUs, both systems converge to similar performance, as the increasing `alltoall` latency (as in § 5.2) becomes the dominant bottleneck at this scale.

5.4 Analysis Results

5.4.1 How does PFT and the padding-free pipeline bring benefits? We evaluate the benefits of PFT format and the associated padding-free pipeline in two dimensions: (1) reduced layer-wise execution time, and (2) improved memory efficiency.

MoE layer time breakdown. To evaluate the impact of PFT, we compare the MoE layer time breakdown of X-MoE and DeepSpeed-MoE when training the *Small* model (EP=8) and *Large* model (EP=64) on 256 GPUs. We disable other optimizations such as RBD to isolate the contribution of PFT. Fig. 11 shows the time comparison in an MoE layer: gating, buffer dispatching, dispatch `alltoall`, expert computation, combine `alltoall`, and buffer combining.

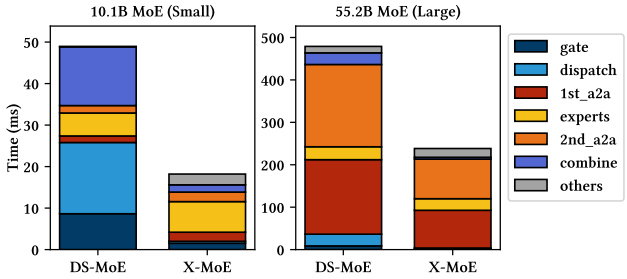


Figure 11: Forward MoE layer time breakdown comparison between DeepSpeed-MoE and X-MoE of training *Small* model and *Large* Model.

The latency reduction arises for different reasons in the *Small* and *Large* models. For the *Small* model, a major inefficiency of the baseline comes from the inefficient gating and buffer dispatch/buffer combine. X-MoE improves on these stages due to the PFT sparse structure and efficient Triton kernels. Specifically, the gating, buffer dispatch and buffer combine stages are accelerated by 5.7×, 35.7× and 8.1× respectively. Note that the expert computation time is slightly increased in X-MoE at this scale. The reason is that X-MoE applies a sequential GEMM on the uneven token buffer, which requires extra data transformations to get the expert input. Despite this overhead, the overall layer time is reduced by 62.3%. For the *Large* model, the largest latency reduction comes from the `alltoall`. X-MoE significantly reduces this time by 50.7% by eliminating zero-padding. The gating, buffer dispatch and combine time are also negligible after X-MoE's optimizations.

Activation memory savings. We compare the per-layer activation memory usage of DeepSpeed-MoE, Tutel, and X-MoE when training the *Large* model on 256 GPUs, using EP=64 and ZeRO-style data parallelism. We report the maximum memory usage across all ranks. As shown in Table 4, X-MoE achieves significantly lower memory consumption than both DeepSpeed-MoE and Tutel, because it reduces memory wastage on dispatching metadata as well as the unused tokens through its padding-free pipeline. Besides, another reason for Tutel's high memory usage is that the Tutel kernel forces the use of float32 on $A_{\texttt{combine}}$ on AMD GPUs.

	DS-MoE	Tutel	X-MoE	Theoretical
Memory (GB)	2.81	1.95	1.21	1.125

Table 4: The activation memory consumption per-MoE-layer.

5.4.2 How does RBD reduce MoE dispatching latency? We evaluate the performance impact of dispatching with and without RBD, with PFT format and padding-free pipeline enabled. We conduct the experiment using a single MoE layer from the *Large* model on 32 GPUs with EP=32. In this setting, the measured redundancy rate is 54.8%. Fig. 12 shows that inter-node `alltoall` communication (shadowed area) dominates the total dispatch time in the padding-free training pipeline. RBD reduces the inter-node communication time by 52.5% by bypassing redundant tokens transferred through the low-bandwidth inter-node links. Although RBD introduces extra overhead, such as intra-node `alltoall` (yellow) and data transformation costs, they are relatively minor compared to the

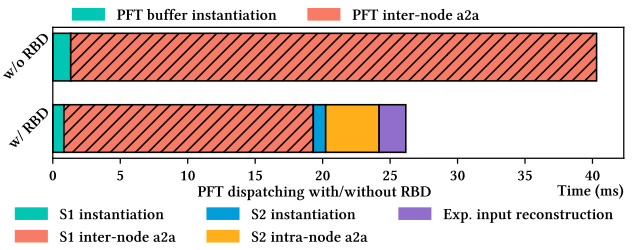


Figure 12: Dispatching time breakdown: With and without RBD under PFT-based dispatching.

savings from reduced inter-node data transfer volume, resulting in an overall performance speedup of 1.55x.

5.4.3 How does SSMB save memory? We evaluate the memory saving benefits of X-MoE by comparing X-MoE with SSMB enabled and X-MoE that uses the conventional tensor-expert-data parallelism (TP+EP+DP) without sequence sharding in MoE blocks, on the *Large* model across 256 GPUs. We enable ZeRO-1 DP and set EP=64 while varying the TP degree from 1 to 4. Fig. 13 shows that enabling SSMB leads to significantly lower memory usage, and the benefit grows as the TP degree increases. This is because SSMB shards sequences within MoE blocks, effectively addressing the shifted memory bottleneck in expert-specialized MoEs. As model size increases, the TP degree naturally grows for non-MoE blocks, making SSMB increasingly important for high memory efficiency for MoE training at scale.

5.4.4 How does SSMB compare to activation checkpointing? One may ask how SSMB compares to activation checkpointing, a technique that reduces memory usage. As shown in Fig. 14, under similar memory savings, X-MoE with SSMB achieves higher throughput. This is because SSMB reduces activation memory at the cost of recomputation and extra `alltoall` during backward pass.

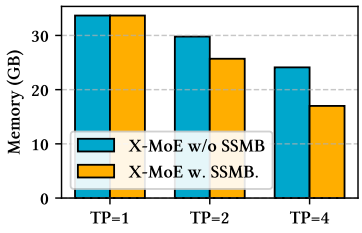


Figure 13: X-MoE's maximum allocated memory across GPUs w/ and w/o SSMB.

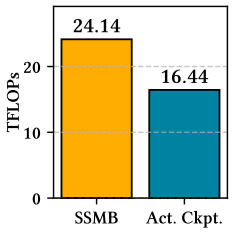


Figure 14: TFLOPs of enabling SSMB vs. activation checkpointing.

5.5 Implementation Validation

To verify the correctness of X-MoE, we compare its training loss curve against DeepSpeed-MoE on the 10.1B MoE model. The experiment is conducted on 16 GPUs with EP=8 and ZeRO DP enabled. In this setting, we confirm that X-MoE closely tracks the convergence behavior of DeepSpeed-MoE, a production grade-implementation, as shown in Fig. 15. This confirms that X-MoE provides numerical convergence while enabling new system optimizations for scaling MoEs. We also investigate why the two curves do not match exactly and find that it is caused by a subtle difference in token-dropping logic. In DeepSpeed-MoE, a token is dropped from an expert if its routing score is negative, regardless of whether the expert's capacity has been exceeded. In contrast, X-MoE only drops tokens when they exceed expert capacity. As a result, X-MoE retains more tokens per batch, which might lead to its slightly lower loss under the same token consumption budget.

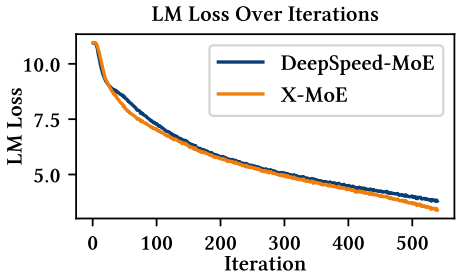


Figure 15: Loss validation with DeepSpeed-MoE and X-MoE.

6 Related Work

MoE Inference Frameworks. SGLang, vLLM, and TensorRT-LLM [12, 21, 26, 39] are general inference frameworks that can also serve MoEs. SGLang provides optimized gather and scatter kernels in triton that are hardware agnostic; however, these kernels leverage block-sparse primitives and incur padding. vLLM provides optimized hardware-agnostic triton kernels via its FlashInfer [37] backend. However, currently only MoEs which activate up to 7B parameters are supported. TensorRT-LLM is NVIDIA's LLM inference engine, but it is tightly coupled to the NVIDIA ecosystem. Moreover, none of these frameworks solve the activation memory explosion of large $A_{dispatch}$ and $A_{combine}$ tensors during MoE training.

Efficient Communication Primitives. DeepEP [38] is an open-source efficient EP implementation by DeepSeek, relying on intrinsics available only on NVIDIA Hopper GPUs. TCCL [19] modifies NVIDIA's NCCL to specifically optimize ring-based collectives on systems where the predominant interconnect is PCIe. Both techniques are tightly coupled to the NVIDIA ecosystem. Centauri [7] introduces an automated way to uncover good schedules where computation is overlapped with communication in heterogeneous environments by decomposing a training task hierarchically into multiple tiers. Unlike these works, X-MoE focuses on system-level optimizations that enable scalable training of expert-specialized MoEs on non-NVIDIA platforms.

7 Conclusion

In this paper, we have taken a leap forward in designing an MoE training system X-MoE to scale expert-specialized MoEs, an increasingly popular model class. With techniques like padding-free MoE training pipeline with cross-platform kernels, redundancy-bypassing dispatching, and hybrid parallelism with sequence sharded MoE blocks, X-MoE enables training of massive MoEs on AMD-based HPC platforms while achieving high throughput, offering a system blueprint to train emerging expert-specialized MoEs on today's HPC platforms.

References

[1] Meta AI. 2024. Introducing Meta LLaMA-3. https://ai.meta.com/blog/meta-llama-3/.

[2] Meta AI. 2025. Llama 4: Multimodal Intelligence. https://ai.meta.com/blog/llama-4-multimodal-intelligence/.

[3] Anthropic. 2024. Claude 3 haiku: our fastest model yet. https://www.anthropic.com/news/claude-3-haiku.

[4] Argonne National Laboratory. 2024. Aurora Supercomputer. https://www.alcf.anl.gov/aurora.

[5] Scott Atchley, Christopher Zimmer, John Lange, and et al. 2023. Frontier: Exploring Exascale. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis, SC 2023.*

[6] Tom Brown, Benjamin Mann, Nick Ryder, and et al. 2020. Language Models are Few-Shot Learners. In *Advances in Neural Information Processing Systems (NeurIPS '20).*

[7] Chang Chen, Xiuhong Li, Qianchao Zhu, Jiangfei Duan, Peng Sun, Xingcheng Zhang, and Chao Yang. 2024. Centauri: Enabling Efficient Scheduling for Communication-Computation Overlap in Large Model Training via Communication Partitioning. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '24).*

[8] Tianqi Chen, Bing Xu, Chiyuan Zhang, and Carlos Guestrin. 2016. Training Deep Nets with Sublinear Memory Cost. arXiv:1604.06174 [cs.LG]

[9] Weihao Cui, Zhenhua Han, Lingji Ouyang, Yichuan Wang, Ningxin Zheng, Lingxiao Ma, Yuqing Yang, Fan Yang, Jilong Xue, Lili Qiu, Lidong Zhou, Quan Chen, Haisheng Tan, and Minyi Guo. 2023. Optimizing Dynamic Neural Networks with Brainstorm. In *USENIX Symposium on Operating Systems Design and Implementation (OSDI '23).* 797–815.

[10] Damai Dai, Chengqi Deng, Chenggang Zhao, R. X. Xu, Huazuo Gao, Deli Chen, Jiashi Li, Wangding Zeng, Xingkai Yu, Y. Wu, Zhenda Xie, Y. K. Li, Panpan Huang, Fuli Luo, Chong Ruan, Zhifang Sui, and Wenfeng Liang. 2024. DeepSeekMoE: Towards Ultimate Expert Specialization in Mixture-of-Experts Language Models. In *the 62nd Annual Meeting of the Association for Computational Linguistics (ACL '24).*

[11] DeepSeek-AI. 2025. DeepSeek-V3 Technical Report. arXiv:2412.19437 [cs.CL]

[12] Zhixu Du, Shiyu Li, Yuhao Wu, Xiangyu Jiang, Jingwei Sun, Qilin Zheng, Yongkai Wu, Ang Li, Hai Helen Li, and Yiran Chen. 2024. SiDA: Sparsity-Inspired Data-Aware Serving for Efficient and Scalable Large Mixture-of-Experts Models. In *Proceedings of Machine Learning and Systems (MLSys '24)*, Vol. 6. 224–238.

[13] William Fedus, Barret Zoph, and Noam Shazeer. 2022. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *The Journal of Machine Learning Research (JMLR '22)* 23, 1 (2022), 5232–5270.

[14] Trevor Gale, Deepak Narayanan, Cliff Young, and Matei Zaharia. 2023. Megablocks: Efficient sparse training with mixture-of-experts. *Proceedings of Machine Learning and Systems (MLSys '23)* 5 (2023).

[15] Jiaao He, Jidong Zhai, Tiago Antunes, Haojie Wang, Fuwen Luo, Shangfeng Shi, and Qin Li. 2022. FasterMoE: modeling and optimizing training of large-scale dynamic pre-trained models. In *Proceedings of the 27th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP '22).* 15 pages.

[16] Changho Hwang, Wei Cui, Yifan Xiong, Ziyue Yang, Ze Liu, Han Hu, Zilong Wang, Rafael Salas, Jithin Jose, Prabhat Ram, et al. 2023. Tutel: Adaptive mixture-of-experts at scale. *Proceedings of Machine Learning and Systems (MLSys '23)* (2023).

[17] Albert Q Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, et al. 2024. Mixtral of experts. *arXiv:2401.04088* (2024).

[18] Chenyu Jiang, Ye Tian, Zhen Jia, Shuai Zheng, Chuan Wu, and Yida Wang. 2024. Lancet: Accelerating mixture-of-experts training via whole graph computation-communication overlapping. (2024).

[19] Heehoon Kim, Junyeol Ryu, and Jaejin Lee. 2024. TCCL: Discovering Better Communication Paths for PCIe GPU Clusters *(ASPLOS '24).*

[20] John Kim, William J. Dally, Steve Scott, and Dennis Abts. 2008. Technology-Driven, Highly-Scalable Dragonfly Topology. In *35th International Symposium on Computer Architecture (ISCA 2008).* IEEE Computer Society, 77–88.

[21] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles, SOSP 2023.* 611–626.

[22] Dmitry Lepikhin, HyoukJoong Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang, Maxim Krikun, Noam Shazeer, and Zhifeng Chen. 2020. Gshard: Scaling giant models with conditional computation and automatic sharding. *arXiv preprint arXiv:2006.16668* (2020).

[23] Dmitry Lepikhin, HyoukJoong Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang, Maxim Krikun, Noam Shazeer, and Zhifeng Chen. 2020. GShard: Scaling Giant Models with Conditional Computation and Automatic Sharding. *CoRR* abs/2006.16668 (2020). arXiv:2006.16668

[24] Aixin Liu, Bei Feng, Bin Wang, Bingxuan Wang, Bo Liu, Chenggang Zhao, Chengqi Dengr, Chong Ruan, Damai Dai, Daya Guo, et al. 2024. Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model. *arXiv preprint arXiv:2405.04434* (2024).

[25] Deepak Narayanan, Mohammad Shoeybi, Jared Casper, Patrick LeGresley, Mostofa Patwary, Vijay Korthikanti, Dmitri Vainbrand, Prethvi Kashinkunti, Julie Bernauer, Bryan Catanzaro, Amar Phanishayee, and Matei Zaharia. 2021. Efficient large-scale language model training on GPU clusters using megatron-LM. In *International Conference for High Performance Computing, Networking, Storage and Analysis, (SC '21).* 58.

[26] NVIDIA Corporation. 2023. *NVIDIA TensorRT: Programmable Inference Accelerator.* https://developer.nvidia.com/tensorrt

[27] OpenAI. 2023. GPT-4 Technical Report. *CoRR* abs/2303.08774 (2023).

[28] OpenAI. 2024. GPT-4o System Card. arXiv:2410.21276 [cs.CL] https://arxiv.org/abs/2410.21276

[29] Myle Ott, Sergey Edunov, Alexei Baevski, Angela Fan, Sam Gross, Nathan Ng, David Grangier, and Michael Auli. 2019. fairseq: A Fast, Extensible Toolkit for Sequence Modeling. In *Proceedings of NAACL-HLT 2019: Demonstrations.*

[30] Samyam Rajbhandari, Conglong Li, Zhewei Yao, Minjia Zhang, Reza Yazdani Aminabadi, Ammar Ahmad Awan, Jeff Rasley, and Yuxiong He. 2022. DeepSpeed-MoE: Advancing Mixture-of-Experts Inference and Training to Power Next-Generation AI Scale. In *International Conference on Machine Learning (ICML'22).*

[31] Jeff Rasley, Samyam Rajbhandari, Olatunji Ruwase, and Yuxiong He. 2020. DeepSpeed: System Optimizations Enable Training Deep Learning Models with Over 100 Billion Parameters. In *The 26th ACM SIGKDD Conference on Knowledge Discovery and Data Mining(KDD 20).* 3505–3506.

[32] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. 2017. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. *arXiv preprint arXiv:1701.06538* (2017).

[33] Siddharth Singh, Olatunji Ruwase, Ammar Ahmad Awan, Samyam Rajbhandari, Yuxiong He, and Abhinav Bhatele. 2023. A hybrid tensor-expert-data parallelism approach to optimize mixture-of-experts training. In *Proceedings of the 37th International Conference on Supercomputing (SC '23).* 203–214.

[34] Gemini Team, Rohan Anil, Sebastian Borgeaud, Jean-Baptiste Alayrac, Jiahui Yu, Radu Soricut, Johan Schalkwyk, Andrew M Dai, Anja Hauth, Katie Millican, et al. 2023. Gemini: a family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805* (2023).

[35] Philippe Tillet, Hsiang-Tsung Kung, and David D. Cox. 2019. Triton: an intermediate language and compiler for tiled neural network computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages (SIGPLAN 2019).*

[36] XAI. 2025. *Grok.* https://x.ai/grok

[37] Zihao Ye, Lequn Chen, Ruihang Lai, Wuwei Lin, Yineng Zhang, Stephanie Wang, Tianqi Chen, Baris Kasikci, Vinod Grover, Arvind Krishnamurthy, and Luis Ceze. 2025. FlashInfer: Efficient and Customizable Attention Engine for LLM Inference Serving. *arXiv preprint arXiv:2501.01005* (2025).

[38] Chenggang Zhao, Shangyan Zhou, Liyue Zhang, Chengqi Deng, Zhean Xu, Yuxuan Liu, Kuai Yu, Jiashi Li, and Liang Zhao. 2025. DeepEP: an efficient expert-parallel communication library. https://github.com/deepseek-ai/DeepEP.

[39] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Jeff Huang, Chuyue Sun, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark W. Barrett, and Ying Sheng. 2023. Efficiently Programming Large Language Models using SGLang. *CoRR* abs/2312.07104 (2023).